PROGRAMACIÓN APLICADA · DESARROLLO MÓVIL

Informe de Evidencias y Defensa Técnica · Guía 03

Pantallas y navegación profesional con Expo Router en EquipRent Mobile

Proyecto	EquipRent Mobile (React Native + TypeScript + Expo)	**Fecha de entrega**	Noviembre 2026
Entidad Padre (1)	UnidadEquipo (id, codigoInventario, serie, modelo)	**Entidad Hija (N)**	Mantenimiento (unidadEquipoId, costo, tipo)

1. Auditoría Estática de Tipos (TypeScript Compilación Limpia)

Conforme al Paso 25 de la Guía 03, se ejecutó la verificación estática estricta con el empaquetador Metro activo. El chequeo valida que no existan módulos sin declarar, contratos de componentes rotos ni inconsistencias de rutas tipadas (código de salida 0).

Comando ejecutado: `npx tsc --noEmit` en la raíz `equiprent-mobile`

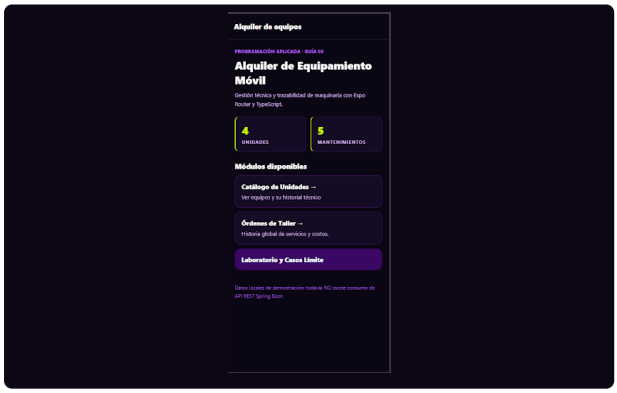
```
PS C:\Users\ASUS\EquipRent\equiprent-mobile> npx tsc --noEmit
PS C:\Users\ASUS\EquipRent\equiprent-mobile>
```

2. Evidencias de Navegación del Dominio y Relación 1:N

Verificación funcional en dispositivo Android con la aplicación Expo Go, demostrando la pila de pantallas, resolución por ID y desacoplamiento.

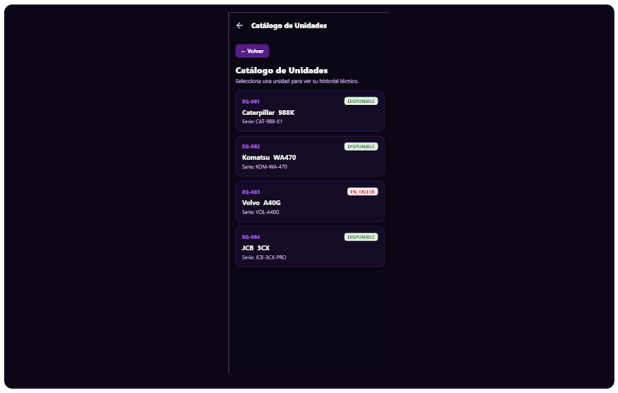
Evidencia 01: Inicio con Métricas

Ruta `/` con contadores reactivos (4 Unidades, 5 Mantenimientos) y enlaces declarativos.



Evidencia 02: Catálogo Padre

Ruta `/unidades` renderizada con FlatList y cabecera nativa con botón de retorno.



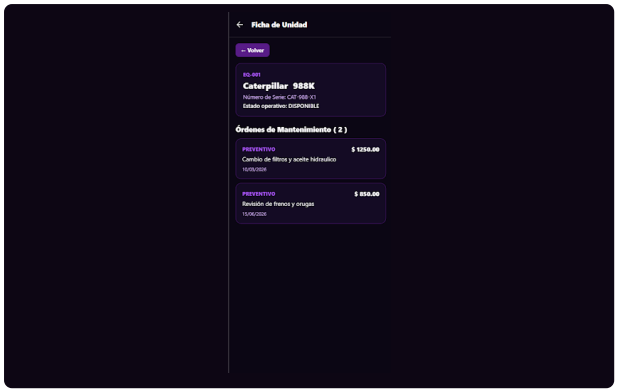
3. Detalle 1:N y Manejo de Casos Límite

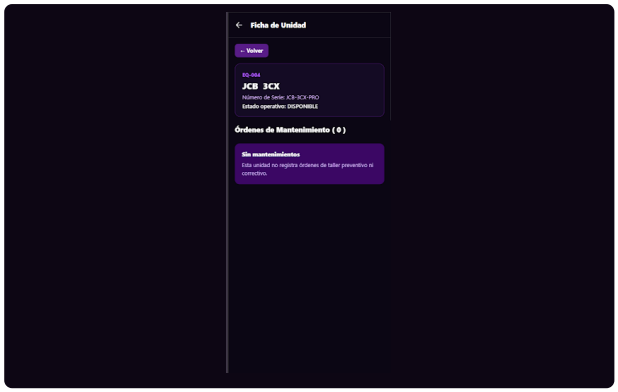
Evidencia 03: Unidad con Hijos

Ruta `/unidades/1`: Ficha de Caterpillar 988K con sus 2 mantenimientos preventivos.

Evidencia 04: Estado Vacío (0 Hijos)

Ruta `/unidades/4`: Ficha de JCB 3CX mostrando EmptyState controlado sin excepciones.

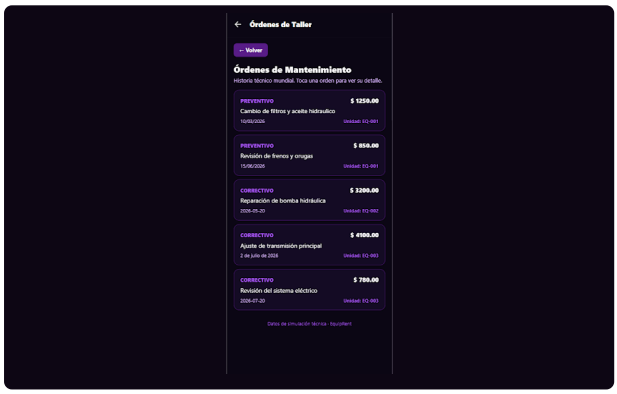




4. Navegación Cruzada y Tolerancia a Fallos

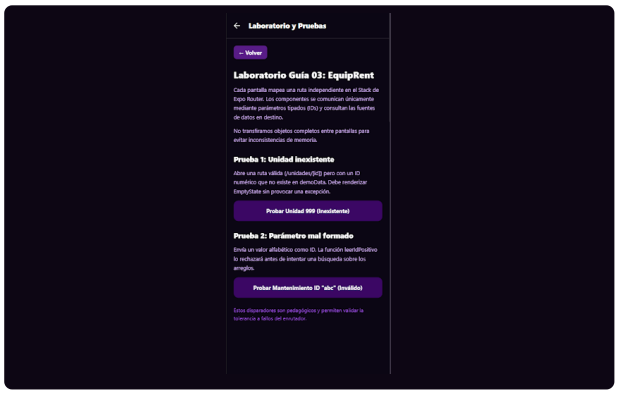
Evidencia 05: Catálogo de Mantenimientos

Ruta `/mantenimientos`: Historial global con resolución de código de unidad por clave foránea.



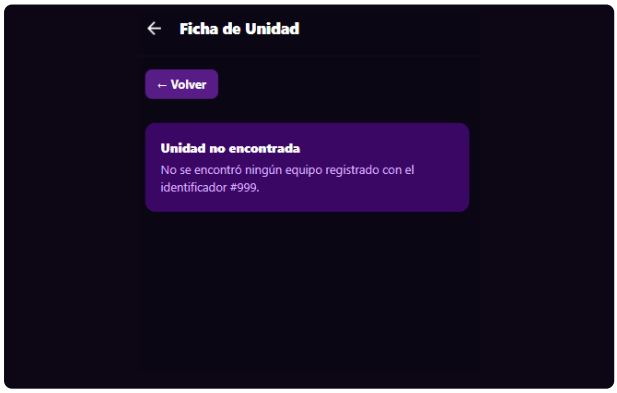
Evidencia 06: Panel de Laboratorio

Ruta `/ayuda`: Disparadores pedagógicos para probar IDs inexistentes y alfabéticos.



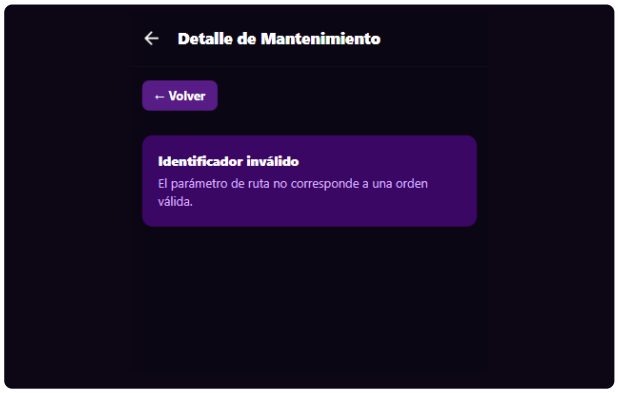
Evidencia 07: Control de ID Inexistente

Ruta `/unidades/999`: ID sintácticamente válido pero sin registro. Muestra EmptyState.



Evidencia 08: Control de ID Mal Formado

Ruta `/mantenimientos/abc`: Rechazado por `leerIdPositivo` antes de consultar datos.



5. Banco de Preguntas y Respuestas para Defensa Oral (Capítulo 16)

P1. ¿Por qué navegamos pasando únicamente el ID numérico y no el objeto completo de la Unidad?
Respuesta técnica: Pasar objetos enteros en los parámetros de navegación sobrecarga el heap de memoria, puede desincronizar los datos si el registro muta en otra pantalla y rompe el patrón de deep linking. Al transferir únicamente el ID, cada pantalla actúa como un cliente autónomo que consulta la fuente de verdad en destino (actualmente `demoData.ts` y próximamente la API REST Spring Boot).

P2. ¿Qué diferencia técnica existe entre la pantalla +not-found.tsx y el error de la Unidad 999?
Respuesta técnica: `+not-found.tsx` es un fallback a nivel de enrutador que se activa cuando el segmento de la URL no coincide con ningún archivo físico del árbol `src/app/` (error de ruta). En cambio, `/unidades/999` sí es atendida por `[id].tsx`; el error ocurre a nivel de dominio al ejecutar `UNIDADES.find()` y obtener `undefined` (error de datos).

P3. ¿Por qué UnidadEquipoCard recibe un callback alSeleccionar en vez de invocar useRouter internamente?
Respuesta técnica: Por el principio de responsabilidad única y desacoplamiento. Las tarjetas son componentes de presentación puros. Si se acoplan directamente a `useRouter`, pierden reusabilidad en pantallas donde seleccionarlas deba abrir un modal, asignar una orden o marcar un selector múltiple. Es la pantalla contenedora la que decide el efecto de navegación.

P4. ¿Por qué no es correcto afirmar que /unidades/1 realiza una petición GET /unidades/1 al servidor?
Respuesta técnica: Porque `/unidades/1` es únicamente una ruta declarativa dentro del árbol de Expo Router procesada en el cliente móvil. En esta etapa no existe ningún cliente HTTP (fetch o Axios) ni comunicación de red con el backend de Spring Boot; los datos son recuperados de constantes en memoria local.

P5. ¿Cuál es la función técnica de leerIdPositivo en routeParams.ts?
Respuesta técnica: Los parámetros obtenidos mediante `useLocalSearchParams` ingresan como cadenas de texto (o arreglos de strings). `leerIdPositivo` aplica una expresión regular estricta (`/^\d+$/`) y comprueba `Number.isSafeInteger(id) > 0` para evitar conversiones no deseadas (como NaN con 'abc' o fallos con números negativos) antes de indexar las colecciones.